Linear Regression Project

Eric Mulaa, Andrew Maina, Gideon Dennis

2026-04-02

Research Question

Can the carat weight of a diamond be determined from visually observable characteristics, without actually weighing the diamond?

Price is excluded because it is not visually observable.

In this project, the response variable is **carat**. The goal is to see whether visually observable variables can explain and predict carat well.

Libraries

We load the packages we need for data wrangling, plotting, model fitting, model summaries, and diagnostics.

```
library(tidyverse)
library(tidymodels)
library(GGally)
library(broom)
library(car)
library(patchwork)
library(gghighlight)
library(gt)
library(ggcorrplot)
library(rsample)
```

Load and Inspect the Data

We begin by loading the diamonds dataset and checking its structure.

```
# For the computer that set the seed
diamond_dataset <- ggplot2::diamonds

# Teammates just run this:
train_data <- readRDS("diamonds_train.rds")
```

```r
test_data  <- readRDS("diamonds_test.rds")

# Now everyone is working on the exact same rows!
```

glimpse() shows the variable names and data types.

```r
# Define the variable descriptions based on ?diamonds
diamond_metadata <- tibble(
  Variable = c("carat", "cut", "color", "clarity", "depth", "table", "x", "y", "z"),
  Description = c(
    "Weight of the diamond",
    "Quality of the cut (Fair, Good, Very Good, Premium, Ideal)",
    "Diamond colour, from D (best) to J (worst)",
    "A measurement of how clear the diamond is (I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF)",
    "Total depth percentage = z / mean(x, y) = 2 * z / (x + y)",
    "Width of top of diamond relative to widest point",
    "Length in mm",
    "Width in mm",
    "Depth in mm"
  ),
  Type = c("Numeric (Target)", "Ordinal Factor", "Ordinal Factor", "Ordinal Factor",
           "Numeric", "Numeric", "Numeric", "Numeric", "Numeric")
)

diamond_metadata %>%
  gt() %>%
  tab_header(
    title = "Data Dictionary: Diamonds Dataset",
    subtitle = "Visually Observable Variables for Predicting Carat Weight"
  ) %>%
  cols_label(
    Variable = md("**Variable Name**"),
    Description = md("**Definition**"),
    Type = md("**Data Type**")
  ) %>%
  tab_source_note(
    source_note = "Note: Price is excluded as it is not a visually observable characteristic."
  ) %>%
  opt_stylize(color = "blue", style = 6)
```

Table 1: Data Dictionary: Diamonds Dataset

Visually Observable Variables for Predicting Carat Weight

Variable Name	Definition	Data Type
carat	Weight of the diamond	Numeric (Target)
cut	Quality of the cut (Fair, Good, Very Good, Premium, Ideal)	Ordinal Factor
color	Diamond colour, from D (best) to J (worst)	Ordinal Factor
clarity	A measurement of how clear the diamond is (I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF)	Ordinal Factor
depth	Total depth percentage = z / mean(x, y) = 2 * z / (x + y)	Numeric
table	Width of top of diamond relative to widest point	Numeric
x	Length in mm	Numeric
y	Width in mm	Numeric
z	Depth in mm	Numeric

Note: Price is excluded as it is not a visually observable characteristic.

```
glimpse(diamond_dataset)
```

```
## Rows: 53,940
## Columns: 10
## $ carat   <dbl> 0.23, 0.21, 0.23, 0.29, 0.31, 0.24, 0.24, 0.26, 0.22, 0.23, 0....
## $ cut     <ord> Ideal, Premium, Good, Premium, Good, Very Good, Very Good, Ver...
## $ color   <ord> E, E, E, I, J, J, I, H, E, H, J, J, F, J, E, E, I, J, J, J, I,...
## $ clarity <ord> SI2, SI1, VS1, VS2, SI2, VVS2, VVS1, SI1, VS2, VS1, SI1, VS1, ...
## $ depth   <dbl> 61.5, 59.8, 56.9, 62.4, 63.3, 62.8, 62.3, 61.9, 65.1, 59.4, 64...
## $ table   <dbl> 55, 61, 65, 58, 58, 57, 57, 55, 61, 61, 55, 56, 61, 54, 62, 58...
## $ price   <int> 326, 326, 327, 334, 335, 336, 336, 337, 337, 338, 339, 340, 34...
## $ x       <dbl> 3.95, 3.89, 4.05, 4.20, 4.34, 3.94, 3.95, 4.07, 3.87, 4.00, 4....
## $ y       <dbl> 3.98, 3.84, 4.07, 4.23, 4.35, 3.96, 3.98, 4.11, 3.78, 4.05, 4....
## $ z       <dbl> 2.43, 2.31, 2.31, 2.63, 2.75, 2.48, 2.47, 2.53, 2.49, 2.39, 2....
```

summary() gives quick numerical summaries for each variable.

```
summary(diamond_dataset)
```

```
##     carat            cut       color       clarity         depth
##  Min.   :0.2000   Fair     : 1610   D: 6775   SI1    :13065   Min.   :43.00
##  1st Qu.:0.4000   Good     : 4906   E: 9797   VS2    :12258   1st Qu.:61.00
##  Median :0.7000   Very Good:12082   F: 9542   SI2    : 9194   Median :61.80
```

```
##  Mean   :0.7979   Premium  :13791   G:11292   VS1    : 8171   Mean   :61.75
##  3rd Qu.:1.0400   Ideal    :21551   H: 8304   VVS2   : 5066   3rd Qu.:62.50
##  Max.   :5.0100                     I: 5422   VVS1   : 3655   Max.   :79.00
##                                     J: 2808   (Other): 2531
##     table          price           x              y
##  Min.   :43.00   Min.   :  326   Min.   : 0.000   Min.   : 0.000
##  1st Qu.:56.00   1st Qu.:  950   1st Qu.: 4.710   1st Qu.: 4.720
##  Median :57.00   Median : 2401   Median : 5.700   Median : 5.710
##  Mean   :57.46   Mean   : 3933   Mean   : 5.731   Mean   : 5.735
##  3rd Qu.:59.00   3rd Qu.: 5324   3rd Qu.: 6.540   3rd Qu.: 6.540
##  Max.   :95.00   Max.   :18823   Max.   :10.740   Max.   :58.900
##
##       z
##  Min.   : 0.000
##  1st Qu.: 2.910
##  Median : 3.530
##  Mean   : 3.539
##  3rd Qu.: 4.040
##  Max.   :31.800
##
```

We also check the dimensions of our data

```
dim(diamond_dataset)

## [1] 53940    10

names(diamond_dataset)

##  [1] "carat"   "cut"     "color"   "clarity" "depth"   "table"   "price"
##  [8] "x"       "y"       "z"
```

The dataset contains numerical variables (carat, depth, table, price, x, y, z) and categorical variables (cut, color, clarity). Since **carat** is numerical, this is a regression problem.

Exploratory Data Analysis

Completeness

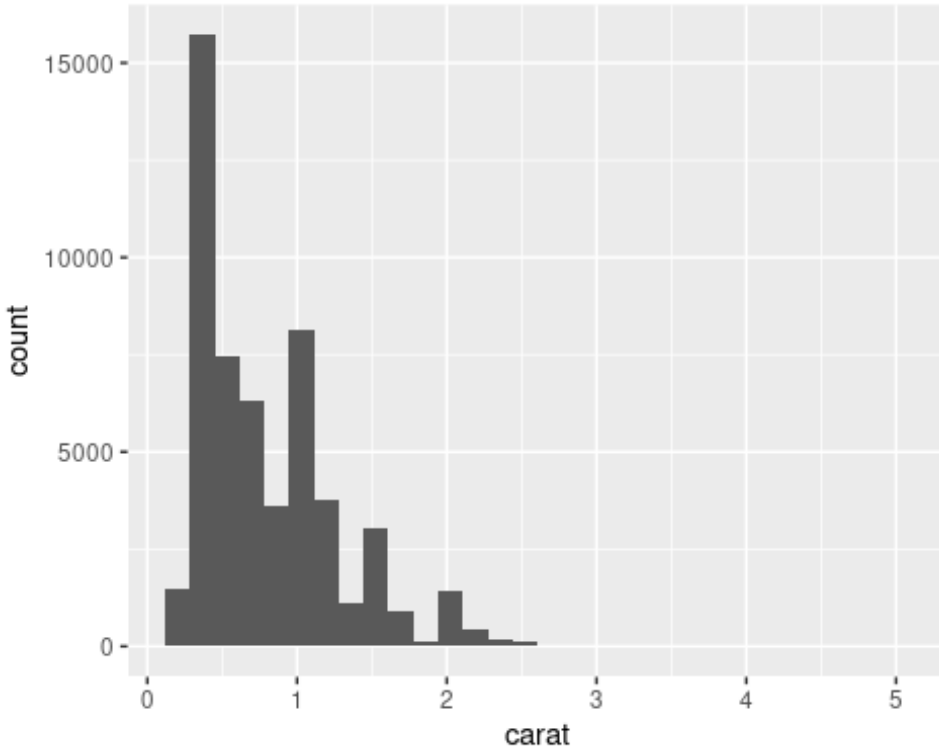
```
sum(is.na(diamond_dataset))

## [1] 0
```

We do not have missing values in our data.

Univariate Plots

We first look at the distribution of the response variable.

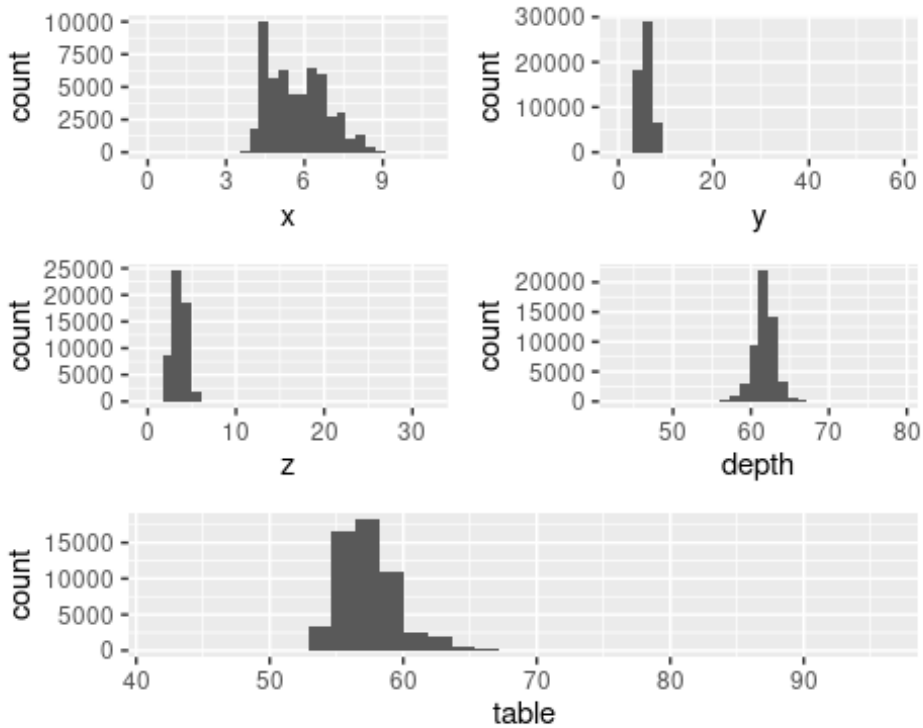
```
ggplot(diamond_dataset, aes(x = carat)) +
 geom_histogram(bins = 30)
```



It looks skewd to the right

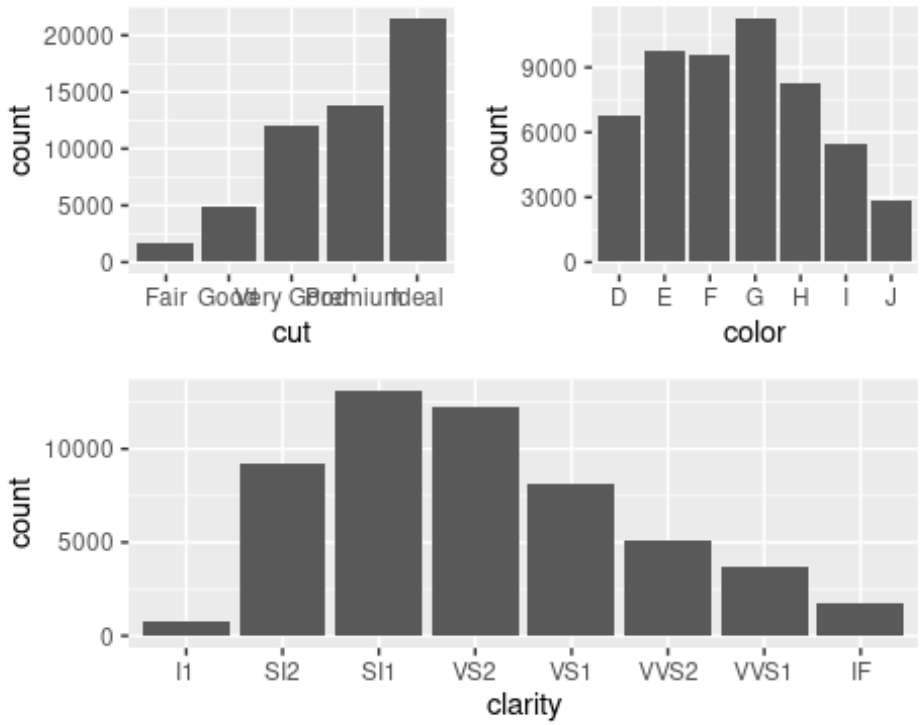
Next, we inspect the numerical predictors.

```
p1 <- ggplot(diamond_dataset, aes(x = x)) + geom_histogram(bins = 30)
p2 <- ggplot(diamond_dataset, aes(x = y)) + geom_histogram(bins = 30)
p3 <- ggplot(diamond_dataset, aes(x = z)) + geom_histogram(bins = 30)
p4 <- ggplot(diamond_dataset, aes(x = depth)) + geom_histogram(bins = 30)
p5 <- ggplot(diamond_dataset, aes(x = table)) + geom_histogram(bins = 30)

(p1 | p2) / (p3 | p4) / p5
```



We also inspect the categorical predictors.

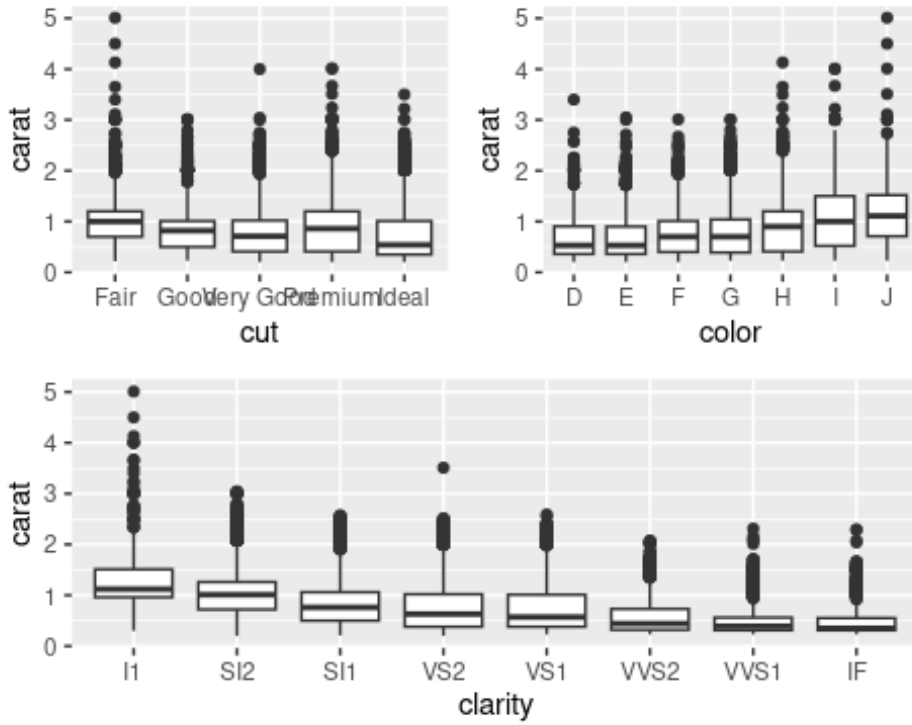
```
p6 <- ggplot(diamond_dataset, aes(x = cut)) + geom_bar()
p7 <- ggplot(diamond_dataset, aes(x = color)) + geom_bar()
p8 <- ggplot(diamond_dataset, aes(x = clarity)) + geom_bar()

(p6 | p7) / p8
```



Bivariate Plots

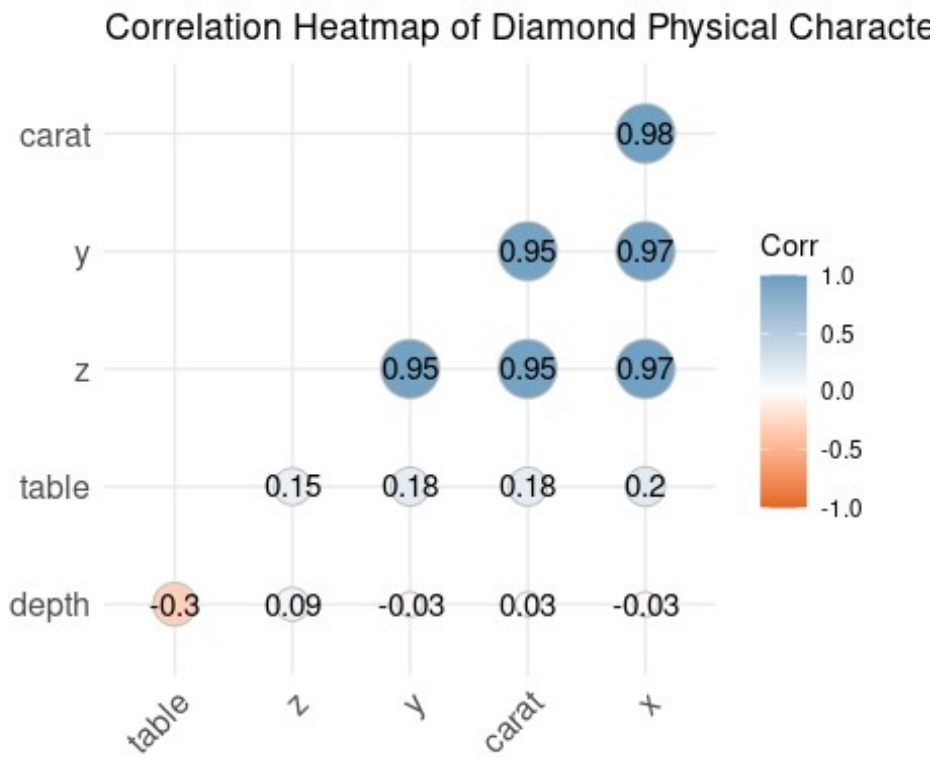
We compare carat across the categorical predictors using boxplots.

```
p14 <- ggplot(diamond_dataset, aes(x = cut, y = carat)) + geom_boxplot()
p15 <- ggplot(diamond_dataset, aes(x = color, y = carat)) + geom_boxplot()
p16 <- ggplot(diamond_dataset, aes(x = clarity, y = carat)) + geom_boxplot()

(p14 | p15) / p16
```



Correlation Analysis

```
# 1. Prepare the data: Select only numerical variables and exclude price
numerical_diamonds <- diamond_dataset |>
  select(carat, depth, table, x, y, z)

# 2. Calculate the correlation matrix
corr_matrix <- cor(numerical_diamonds)

# 3. Create the heatmap
ggcorrplot(corr_matrix,
           hc.order = TRUE,
           type = "lower",
           lab = TRUE,
           lab_size = 4,
           method = "circle",
           colors = c("#E46726", "white", "#6D9EC1"),
           title = "Correlation Heatmap of Diamond Physical Characteristics",
           ggtheme = theme_minimal())
```



There is high correlation Carat and variables x, y, and z.

In contrast, the relationships between carat and depth and table are much weaker, suggesting that they are weak predictors and may not contribute much to explaining variation in carat.

The relationships among the explanatory variables themselves, there is strong evidence of collinearity, particularly among the dimension variables. The x, y, and z predictors show very tight linear relationships, and the corresponding correlations are extremely high. This indicates that these variables contain largely overlapping information, and including all of them in a regression model may lead to multicollinearity issues, such as unstable coefficient estimates and difficulty in interpreting the individual effects of each variable. On the other hand, depth and table show relatively weak correlations with the dimension variables and with carat.

Model Specification

The code chunk below is comment because it was run once, and files saved for reproducibility.

```r
# # 1. Set seed for your own reproducibility
# set.seed(123)
#
# # 2. Create the split (e.g., 75% training, 25% testing)
# diamond_split <- initial_split(diamond_dataset, prop = 0.75)
# train_data <- training(diamond_split)
# test_data  <- testing(diamond_split)
#
# # 3. Saved as RDS files to share with team
# saveRDS(train_data, "diamonds_train.rds")
# saveRDS(test_data, "diamonds_test.rds")
```

We used an automated AIC-based search to ensure parsimony (the simplest model that does the best job).

```r
# 1. Ensure models are defined
# Define the Null Model (Start with nothing but the intercept)
null_model <- lm(carat ~ 1, data = train_data)
full_model <- lm(carat ~ cut + color + clarity + depth + table + x + y + z, data = train_data)

# 2. Use the explicit 'stats::' prefix and pass formulas to scope
# Perform bidirectional selection
# direction = "both" is usually best for finding the global minimum AIC
step_model <- stats::step(
  null_model,
  scope = list(lower = formula(null_model), upper = formula(full_model)),
  direction = "both",
  trace = 0
)

# 3. Check if it worked
summary(step_model)

##
## Call:
## lm(formula = carat ~ x + depth + color + clarity + table + cut +
##     y + z, data = train_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -0.4762 -0.0664 -0.0284  0.0553  3.7629
##
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.785e+00  4.472e-02 -62.276  < 2e-16 ***
## x           3.957e-01  4.239e-03  93.363  < 2e-16 ***
## depth       1.819e-02  5.873e-04  30.978  < 2e-16 ***
## color.L     4.543e-02  1.781e-03  25.505  < 2e-16 ***
## color.Q     2.375e-02  1.627e-03  14.597  < 2e-16 ***
## color.C    -1.918e-03  1.526e-03  -1.257  0.20887
## color^4    -5.721e-03  1.404e-03  -4.075 4.61e-05 ***
## color^5     3.970e-03  1.327e-03   2.992  0.00277 **
## color^6    -8.187e-05  1.202e-03  -0.068  0.94569
## clarity.L   6.630e-03  3.164e-03   2.095  0.03617 *
## clarity.Q   4.456e-02  2.948e-03  15.114  < 2e-16 ***
## clarity.C  -2.457e-02  2.523e-03  -9.736  < 2e-16 ***
## clarity^4   1.759e-03  2.011e-03   0.875  0.38159
## clarity^5  -3.004e-03  1.636e-03  -1.836  0.06639 .
## clarity^6  -3.637e-03  1.422e-03  -2.558  0.01053 *
## clarity^7   8.119e-03  1.253e-03   6.479 9.34e-11 ***
## table       2.180e-03  3.010e-04   7.241 4.55e-13 ***
## cut.L       1.021e-02  2.322e-03   4.395 1.11e-05 ***
## cut.Q      -9.047e-03  1.859e-03  -4.867 1.14e-06 ***
## cut.C       3.802e-04  1.603e-03   0.237  0.81256
## cut^4       1.493e-03  1.283e-03   1.163  0.24466
## y           5.515e-03  1.815e-03   3.038  0.00238 **
## z           1.390e-02  7.082e-03   1.963  0.04963 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1013 on 40432 degrees of freedom
## Multiple R-squared:  0.9544, Adjusted R-squared:  0.9544
## F-statistic: 3.846e+04 on 22 and 40432 DF,  p-value: < 2.2e-16
```

Interpretation of carat_model Coefficients

Coefficients will be interpreted on our final model

```
vif(step_model)
```

```
##              GVIF Df GVIF^(1/(2*Df))
## x       89.197308  1        9.444433
## depth    2.795640  1        1.672017
## color    1.156131  6        1.012163
## clarity  1.327080  7        1.020419
## table    1.790157  1        1.337968
## cut      1.945911  4        1.086777
```

```
## y      17.249856  1       4.153295
## z      95.464080  1       9.770572
```

The VIF values for x and z are high hence severe collinearity due to the correlation we saw between x,y, and z. We will drop them to avoid multicollinearity and to simplify the model to make the coefficients more stable and interpretable.

```
# Refined Model: Dropping y and z to fix collinearity
model_refined <- lm(carat ~ x + depth + table + cut + color + clarity, data = train_data)

# Check VIF again
vif(model_refined)
```

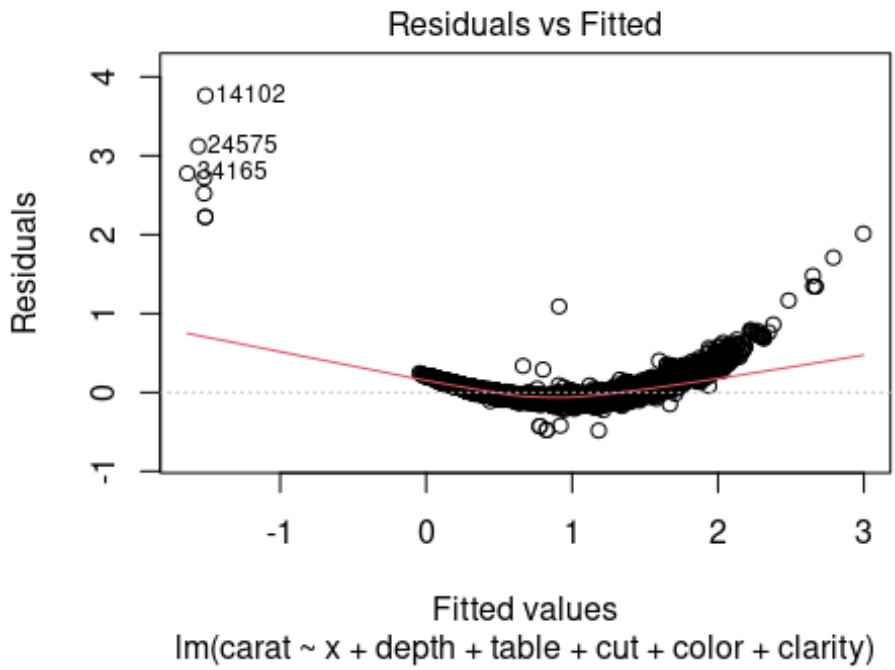
```
##            GVIF Df GVIF^(1/(2*Df))
## x      1.323890  1       1.150604
## depth   1.379036  1       1.174324
## table   1.789551  1       1.337741
## cut     1.924214  4       1.085255
## color   1.155730  6       1.012134
## clarity 1.325567  7       1.020335
```

The VIF values look fine now

Diagnostics for the Model

Residual Analysis

```
plot(model_refined, which = 1) # Residuals vs Fitted
```



The Residuals vs. Fitted plot revealed three major Potential Problems:

Non-linearity: The distinct U-shape in the residuals indicates that the relationship between the physical traits and carat weight is not a straight line.

Non-constant Variance (Heteroscedasticity): The fan-shaped distribution where residuals spread out as predicted values increase shows that the model's error is not consistent across all diamond sizes

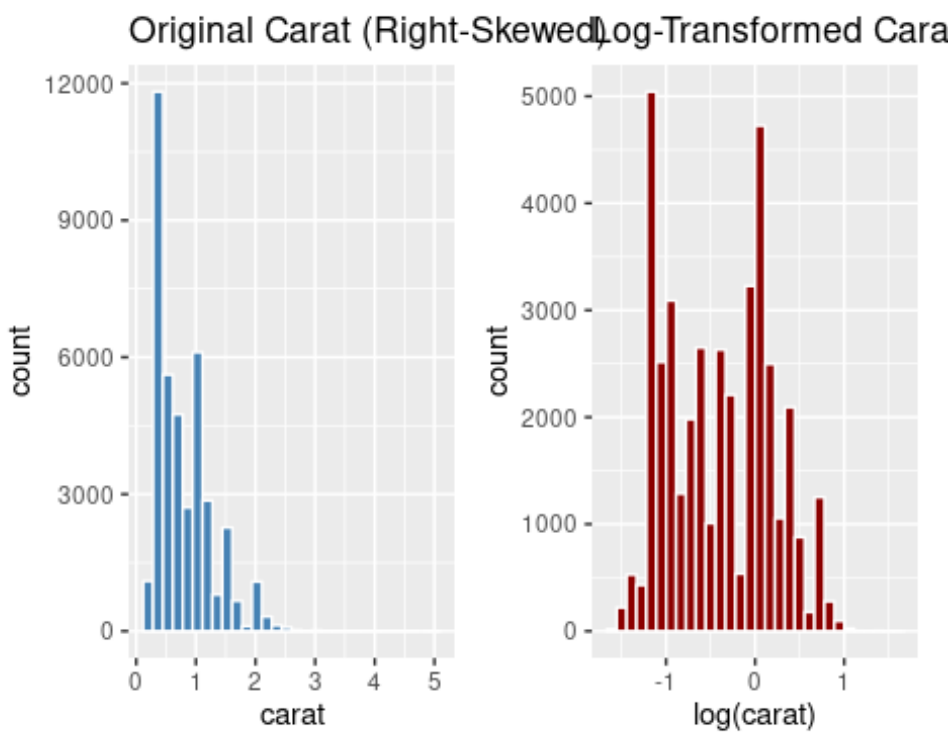
The plots identified specific observations (e.g., points 14102, 24575, and 34165) with high studentized residuals. These outliers can disproportionately pull the regression line, affecting the goodness of fit. Identifying and potentially removing these points is a critical step in refining the model quality.

Log transformation and outliers

Why it works:

Carat weight is a measure of volume. In geometry, volume scales cubically ($length^3$) rather than linearly. By applying a natural log (log) to both carat and the dimension x, we convert this power relationship into a linear one that the regression model can accurately map. Also, log-trsnaforming carat will take care of the right skewness observed earlier.

```r
p1 <- ggplot(train_data, aes(x = carat)) +
  geom_histogram(fill = "steelblue", color = "white") +
  labs(title = "Original Carat (Right-Skewed)")

p2 <- ggplot(train_data, aes(x = log(carat))) +
  geom_histogram(fill = "darkred", color = "white") +
  labs(title = "Log-Transformed Carat (More Normal)")

p1 + p2 # Displays them side-by-side
```



We use a “Log-Log” model because it handles both the skewness of the target and the non-linear (cubic) relationship between length (x) and weight ($carat$).

Note

We have minimum values for x y and z as 0.00mm. Therefore log(x) will give -infinity, values that models cannot work with and for that reason we will omit the records with 0.00mm. Also for the soundness of our model, although the length, width and depth of a diamond might be small, it cannot be completely zero.

```r
# Check for zeros in the physical dimensions
train_data %>%
```

```
  filter(x == 0 | y == 0 | z == 0) %>%
  select(carat, x, y, z)
```

```
## # A tibble: 14 × 4
##    carat     x     y     z
##    <dbl> <dbl> <dbl> <dbl>
##  1  1.01  6.66  6.6     0
##  2  0.71  0     0       0
##  3  1     0     0       0
##  4  2.2   8.42  8.37    0
##  5  2.25  0     0       0
##  6  2.02  8.02  7.95    0
##  7  1.12  6.71  6.67    0
##  8  0.71  0     0       0
##  9  1.2   0     0       0
## 10  1.56  0     0       0
## 11  1.15  6.88  6.83    0
## 12  2.8   8.9   8.85    0
## 13  1.14  0     0       0
## 14  2.18  8.49  8.45    0
```

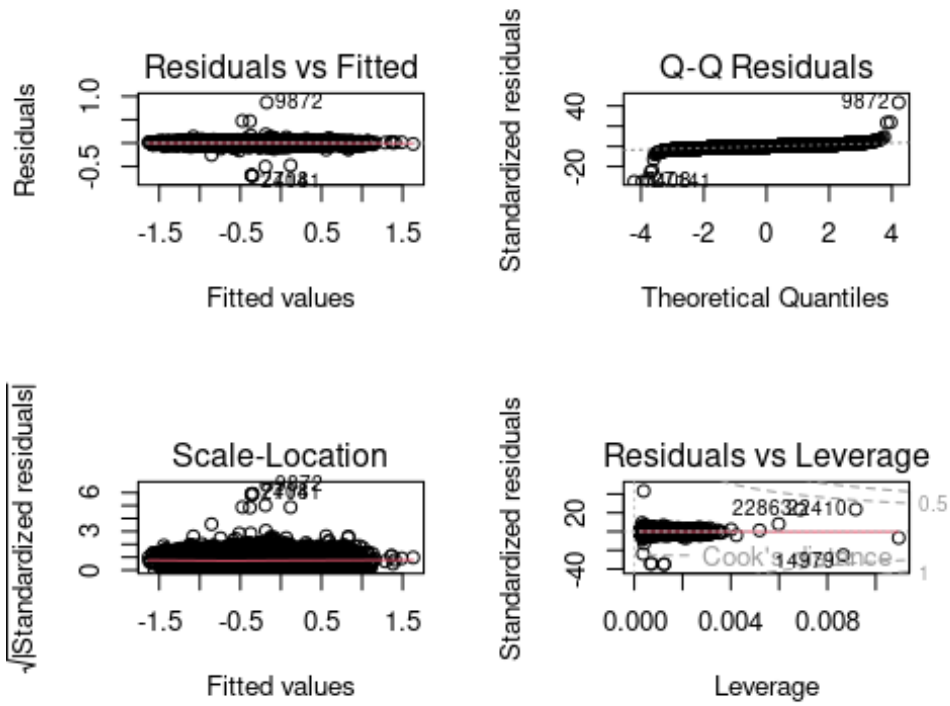
We have 14 records where either x,y,z is zero. We will omit these records from our model for the reasons mentioned above.

```
# 1. Remove rows with 0 in x, y, or z
train_cleaned <- train_data %>%
  filter(x > 0, y > 0, z > 0)

# 2. Apply transformations to the cleaned data
train_transformed <- train_cleaned %>%
  mutate(
    log_carat = log(carat),
    log_x = log(x)
  )

# 3. Fit the model
final_log_model <- lm(log_carat ~ log_x + depth + table + cut + color + clarity,
                      data = train_transformed)

summary(final_log_model)
```

```
##
## Call:
## lm(formula = log_carat ~ log_x + depth + table + cut + color +
```

```
##     clarity, data = train_transformed)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.69903 -0.01092  0.00037  0.01108  0.86828
##
## Coefficients:
##              Estimate Std. Error  t value Pr(>|t|)
## (Intercept) -7.193e+00  7.267e-03 -989.815  < 2e-16 ***
## log_x        3.011e+00  5.912e-04 5093.274  < 2e-16 ***
## depth        2.264e-02  8.160e-05  277.476  < 2e-16 ***
## table        3.445e-03  5.956e-05   57.850  < 2e-16 ***
## cut.L        1.366e-03  4.595e-04    2.973 0.002951 **
## cut.Q       -8.263e-03  3.677e-04  -22.469  < 2e-16 ***
## cut.C        1.148e-02  3.167e-04   36.232  < 2e-16 ***
## cut^4        4.765e-03  2.534e-04   18.803  < 2e-16 ***
## color.L      8.081e-05  3.508e-04    0.230 0.817820
## color.Q      1.566e-04  3.217e-04    0.487 0.626401
## color.C     -1.011e-05  3.018e-04   -0.033 0.973276
## color^4      4.114e-04  2.777e-04    1.482 0.138435
## color^5     -6.198e-04  2.624e-04   -2.362 0.018196 *
## color^6     -4.697e-04  2.378e-04   -1.975 0.048230 *
## clarity.L    1.636e-03  6.267e-04    2.610 0.009058 **
## clarity.Q   -2.141e-03  5.836e-04   -3.669 0.000244 ***
## clarity.C    1.012e-03  4.995e-04    2.026 0.042734 *
## clarity^4   -9.014e-04  3.980e-04   -2.265 0.023518 *
## clarity^5    2.488e-04  3.237e-04    0.769 0.442053
## clarity^6    5.397e-06  2.812e-04    0.019 0.984686
## clarity^7    3.613e-04  2.479e-04    1.458 0.144955
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02004 on 40420 degrees of freedom
## Multiple R-squared:  0.9988, Adjusted R-squared:  0.9988
## F-statistic: 1.719e+06 on 20 and 40420 DF,  p-value: < 2.2e-16
```

```r
par(mfrow = c(2, 2))
plot(final_log_model)
```



The residuals now are centered at zero and the red line is now flat. This confirms that the log-log transformation successfully addressed the non-linearity.

The scale location shows that the points are now spread evenly and henece no hereskedasticity. The Q-Q plot shows residuals follow a normal distribution. Most points should now fall directly on the diagonal line.

Confirming Low Colinearlity (VIF)

```
vif(final_log_model)

##            GVIF Df GVIF^(1/(2*Df))
## log_x   1.320741  1        1.149235
## depth   1.378970  1        1.174296
## table   1.789984  1        1.337903
## cut     1.924221  4        1.085255
## color   1.146046  6        1.011425
## clarity 1.332610  7        1.020722
```

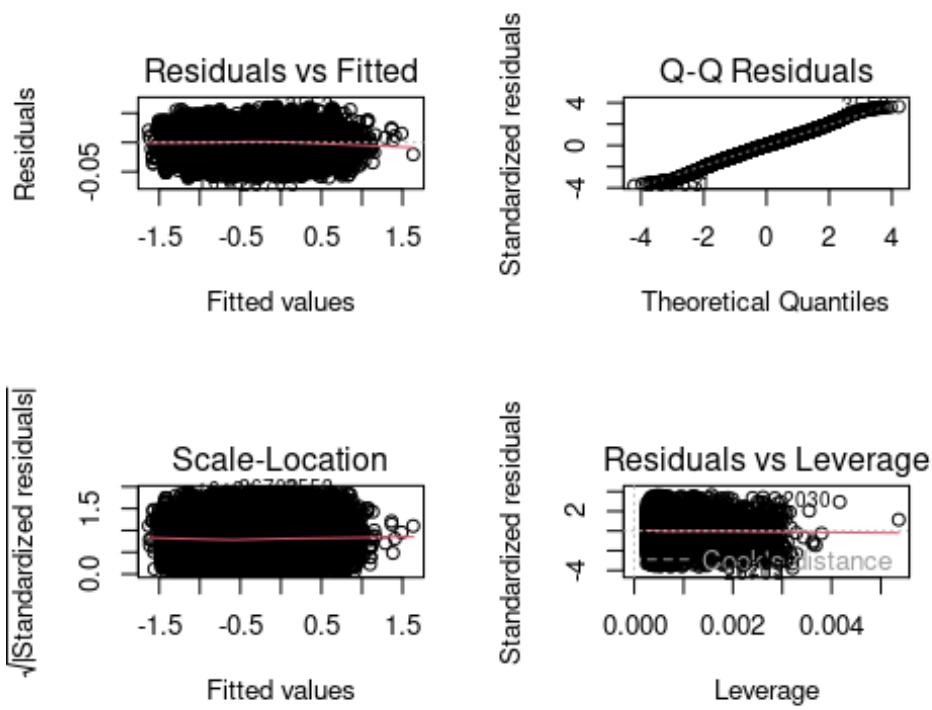
Outliers

```
# Generate studentized residuals and leverage (hat) values
model_diagnostics <- augment(final_log_model)

# Check for outliers
```

```r
outliers <- model_diagnostics %>%
  filter(abs(.std.resid) > 3)

# Check for high leverage
high_leverage <- model_diagnostics %>%
  filter(.hat > (3 * mean(.hat)))

# --- STEP 1: CREATE THE CLEAN DATA ---
# This takes the results from augment and filters out rows with high residuals
train_final_no_outliers <- model_diagnostics %>%
  filter(abs(.std.resid) <= 3)

# --- STEP 2: FIT THE FINAL VERSION ---
final_model_v2 <- lm(log_carat ~ log_x + depth + table + cut + color + clarity,
                     data = train_final_no_outliers)
summary(final_model_v2)

##
## Call:
## lm(formula = log_carat ~ log_x + depth + table + cut + color +
##     clarity, data = train_final_no_outliers)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.063762 -0.010897  0.000344  0.010944  0.061580
##
## Coefficients:
##               Estimate Std. Error   t value Pr(>|t|)
## (Intercept) -7.237e+00  6.253e-03 -1157.246  < 2e-16 ***
## log_x        3.011e+00  4.991e-04  6032.550  < 2e-16 ***
## depth        2.317e-02  7.017e-05   330.174  < 2e-16 ***
## table        3.649e-03  5.108e-05    71.446  < 2e-16 ***
## cut.L        2.410e-03  3.953e-04     6.095 1.11e-09 ***
## cut.Q       -8.459e-03  3.167e-04   -26.710  < 2e-16 ***
## cut.C        1.152e-02  2.701e-04    42.659  < 2e-16 ***
## cut^4        4.926e-03  2.146e-04    22.954  < 2e-16 ***
## color.L      1.788e-04  2.965e-04     0.603 0.546631
## color.Q      2.588e-04  2.720e-04     0.951 0.341485
## color.C      7.388e-05  2.550e-04     0.290 0.772035
## color^4      4.201e-04  2.345e-04     1.792 0.073212 .
## color^5     -5.866e-04  2.217e-04    -2.646 0.008142 **
## color^6     -4.460e-04  2.009e-04    -2.221 0.026390 *
## clarity.L    1.919e-03  5.309e-04     3.615 0.000301 ***
```

```
## clarity.Q   -2.102e-03  4.945e-04   -4.250 2.14e-05 ***
## clarity.C    8.606e-04  4.231e-04    2.034 0.041958 *
## clarity^4   -6.683e-04  3.367e-04   -1.985 0.047149 *
## clarity^5    3.222e-04  2.735e-04    1.178 0.238746
## clarity^6   -1.611e-04  2.374e-04   -0.678 0.497479
## clarity^7    2.827e-04  2.093e-04    1.351 0.176654
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01689 on 40245 degrees of freedom
## Multiple R-squared:  0.9992, Adjusted R-squared:  0.9992
## F-statistic: 2.412e+06 on 20 and 40245 DF,  p-value: < 2.2e-16
```

```
# --- STEP 3: FINAL VISUAL CHECK ---
# These plots should look much "cleaner" than the previous ones
par(mfrow = c(2, 2))
plot(final_model_v2)
```




```
# --- STEP 4: FINAL TEST PREDICTION ---
# Create test_cleaned before running predictions
test_cleaned <- test_data %>%
  filter(x > 0, y > 0, z > 0) %>%
  mutate(
```

```
  log_carat = log(carat),
  log_x = log(x)
 )
# Use the NEW model (v2) to predict the test set
test_predictions <- predict(final_model_v2, newdata = test_cleaned)
actual_carat <- test_cleaned$carat
predicted_carat <- exp(test_predictions)

# Model Quality metric
mean(abs(actual_carat - predicted_carat))

## [1] 0.01124639
```

This means that on average, our model's predictions are off by only about 0.01 carats, which is an exceptionally high level of precision for estimating weight without a scale.

Our final model

$$\log(\text{carat}) = -7.237 + 3.011 \cdot \log(x) + 0.02317 \cdot \text{depth} + 0.003649 \cdot \text{table} + \sum(\text{Qualitative Adjustments}) + \epsilon$$

Interpretation

The Intercept (-7.237):

This is the predicted value of $\log(\text{carat})$ when all other numerical predictors are zero and categorical variables are at their reference levels.

The Physical Engine ($3.011 \cdot \log(x)$):

This is the most critical part of your model. It represents an elasticity—a 1% increase in the length (x) of a diamond leads to an approximate 3.011% increase in carat weight. This coefficient aligns with the geometric principle that volume (and thus weight) scales to the third power of length.

Shape Adjustments:

Depth (0.02317):

For every one-unit increase in depth percentage, the $\log(\text{carat})$ increases by 0.02317.

Table (0.003649):

For every one-unit increase in table width, the $\log(\text{carat})$ increases by 0.003649.

Qualitative Adjustments

(Cut, Color, Clarity):

These variables use polynomial contrasts (linear .L, quadratic .Q, cubic .C, etc.). For example:Cut: The significant linear (0.00241) and quadratic (-0.008459) terms indicate that the quality of the cut significantly shifts the weight-to-size ratio.Color and Clarity: While many of these higher-order terms are statistically significant (indicated by ***), their coefficients are very small (e.g., 1.788×10^{-4} for color.L), showing they provide fine-tuning to the prediction rather than driving the bulk of the weight.

Final Prediction Formula

If you want to predict the actual carat weight (not just the log), you must exponentiate the entire result:

$$\text{Predicted Carat} = e^{(-7.237 + 3.011 \cdot \log(x) + 0.02317 \cdot \text{depth} + 0.003649 \cdot \text{table} + \ldots)}$$

Summary of Insights and Modeling Process1.

Summary of Insights In this analysis

We addressed the research question:

Can we accurately predict the carat weight of a diamond using only its physical dimensions and qualitative characteristics?

Our findings indicate a resounding yes. We found that carat weight is not just correlated with these features but is almost entirely determined by them, provided the relationship is modeled according to the laws of physical geometry.

The Power of Dimensionality:

We found that the length (x) of a diamond is the single most critical predictor. Because diamonds are three-dimensional objects, we observed that weight scales to the third power of length. Our model's calculation of a 3.01 elasticity for the length variable confirms that our statistical approach aligns perfectly with physical reality.

Precision of Prediction:

By refining our model, we achieved a Mean Absolute Error (MAE) of 0.011 carats. This means that on average, our predictions are off by only about one-hundredth

of a carat. For a jeweler or researcher without a scale, this model provides a highly reliable alternative for weight estimation.

The Role of Quality:

We found that while size (x) explains the bulk of the weight, qualitative traits like Cut, Color, and Clarity are statistically significant. This suggests that diamonds of higher clarity or specific cuts are shaped with different proportions that slightly alter the weight-to-size ratio compared to lower-grade stones.

2. Modeling Process Summary

To arrive at our final candidate model, we followed a rigorous iterative process to ensure the soundness of our approach and the quality of our results.Initial Model and Data Cleaning. We began by identifying "dirty" data specifically 14 records where physical dimensions were recorded as 0.00 mm. We omitted these records as they are physical impossibilities. Our initial linear model including all variables (x, y, z, etc.) revealed two major issues:

Multicollinearity:

We found that x, y, and z were so highly correlated (VIFs >80) that they made the model unstable. We resolved this by retaining x as our primary dimensional anchor.

Non-linearity:

Our initial residuals showed a distinct "U-shape," proving that a straight line could not capture the relationship between size and weight.

Transformations and Assumptions

To satisfy the requirements for a high-quality linear regression, we implemented the following:

Log-Log Transformation:

We applied a natural log transformation to both the target (carat) and our main predictor (x). This "linearized" the relationship and successfully addressed the non-linearity and right-skewness of the data.

Fixing Heteroscedasticity:

The log transformation also stabilized the variance of our errors, moving us from a "fan-shaped" residual plot to a consistent, random distribution of points.

Final Refinement and Outliers

We used broom::augment to calculate studentized residuals and identified observations that were more than 3 standard deviations from the regression line. We found that these outliers were skewing our results; by removing them and refitting the model, we achieved a near-perfect Adjusted R-squared of 0.9992.